

Problem Statement

You are working as an ML engineer for an e-commerce company Carsdepo.com, which provides an online marketplace for the purchase and sale of used/new cars.

The company was founded in 2016 as an online platform to buy and sell used/new cars. The company offers end-to-end transactions, where it provides a smooth transition of ownership from the sellers to the buyers at the best rates. It sells more than 1 million cars worldwide annually and is considered one of the top three companies in the Asian market.

While expanding operations internationally into several countries, it noticed that the market for used cars trumps over the new cars. However, this is a complicated problem to solve, as buyers want to be assured of the money they invest while buying.

When a used car is listed, its worth must be effectively determined based on various features. A car can suffer from different types of damages, impacting its original shape and form, and this heavily impacts its resale value.

Therefore, when a used car is bought or sold, its actual market value must be known. Also, It is often impossible to manually inspect every part of a car, as this can be quite labour intensive.

In light of this, the company wants to build an automated damage-detection system that can detect any type of damage in cars. This solution can help determine the true resale value of a car. Based on what you learnt in the previous modules, you

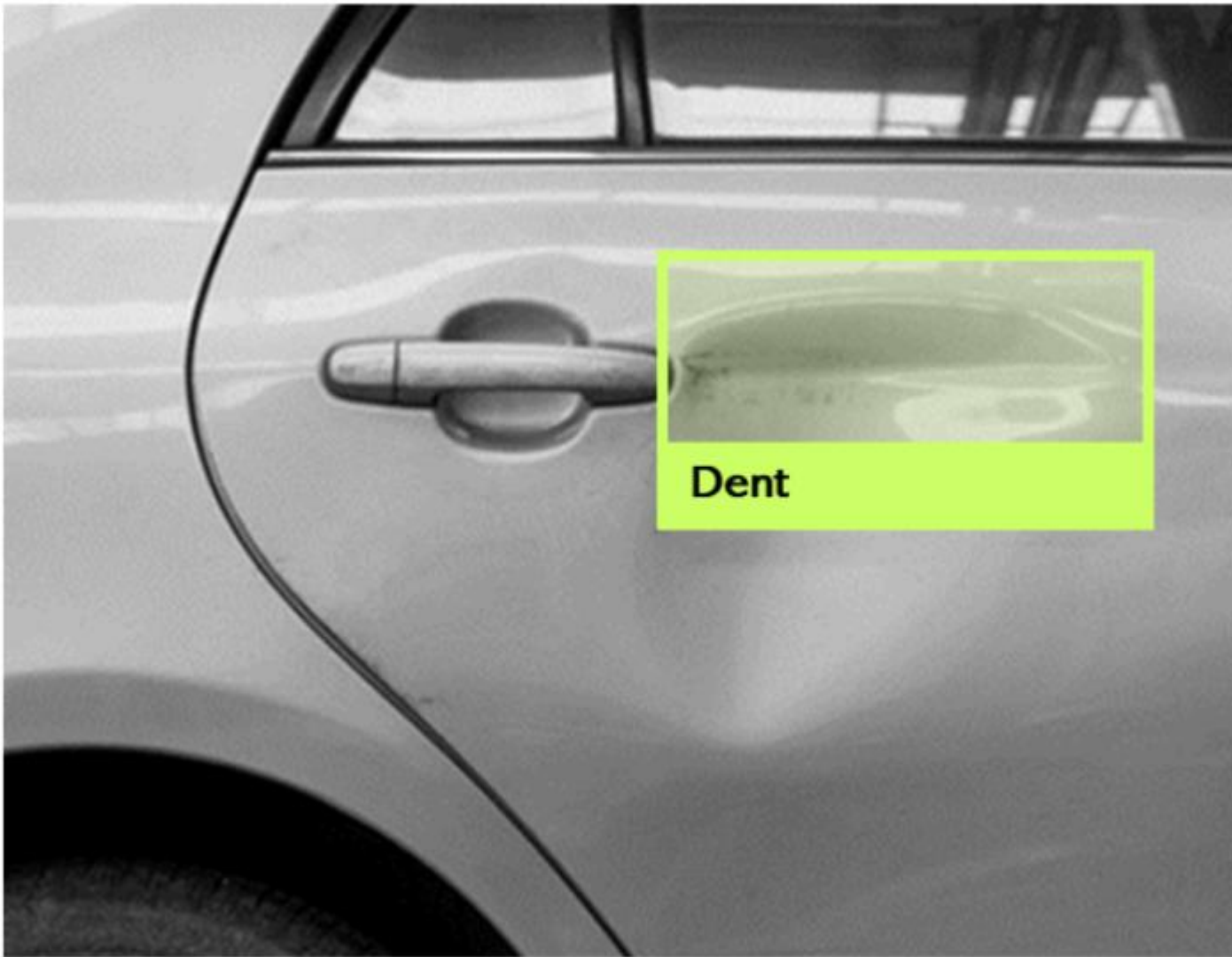
must understand the root problem that the company is trying to solve and build an ML system that provides an effective solution.

Let's take a look at the problem statement:

The company wants to provide an effective pricing strategy for any listing of used cars. Earlier, this problem was solved by a manual inspection conducted by a representative, who would inspect every feature of a used car. This solution was effective, as the representative considered every factor affecting the resale value (especially the damage); however, this solution is not scalable, as the company is growing.

Using the millions of images of used cars the company has collected over the years, you must build an object-detection model that can detect the presence of any damage in cars.

Among the various types of damage, the model should be able to detect two: scratches and dents. It should return "None" if no damage is detected. The product team will use these results to map the resale value of a car based on the type of damage detected.



An image of a car with the label: 'Dent'

Note: You have a large repository of images, where the count of each damage is almost equal. However, some of the images are not yet labelled.

You must create an ML system that has the features of a complete production stack, from experiment tracking to automated model deployment and monitoring. The framework should also be able to train if additional annotations are available later.

Q1. System design: Based on the above information, describe the KPI that the business should track.

Key Performance that the business should track are mentioned below.

Model Performance:

Accuracy of Damage Detection, ratio of correctly predicted instances to the total number of instances.

Precision, Recall, and F1 Score Recall:

Precision: false positives (e.g., predicting damage when there is none) could incorrectly impact the car's resale value.

Recall: Missing damages (false negatives) would undervalue the system's effectiveness.

The ratio of true positive predictions to the actual positives in the data. It measures how well the model captures all relevant instances.

Object Detection Accuracy (mAP - Mean Average Precision):

Standard metric that evaluates how well the model detects and localizes damage (scratches or dents) on the car images.

False Positive and False Negative Rates: High false positives (detecting damage when none exists) can lower user trust in the system, while high false negatives (missing real damage) reduce the system's utility in pricing.

Business Impact

Resale Price Accuracy: A KPI would be how close the automated system's price estimate is to the actual final sale price. Is model helping in reduction in Manual Inspection, its system versus manual representatives? how consistent price value / the model is in comparison to manual inspection.

Customer Satisfaction (Trust Index) : Survey or feedback systems.

System supports business expansion that increase/expand the business.

Operational Efficiency.

Training Time and Resource Utilization, helps in optimizing cost and time.

System Uptime and Reliability. KPI that measures the average cost per image processed can help optimize resource usage.

The company can ensure that its damage detection model not only performs well but also scales efficiently and positively impacts the business.

Q2. System design: Your company has decided to build an MLOps system. What advantages would you get from building an MLOps system rather than a simple model?

Advantages we could get both operational and business goals, particularly in terms of scalability, reliability, and automation. Here are the key benefits:

CICD:

Automation process of entire end-end MLOps cycle. DevOps best practices like **Continuous Integration (CI)** and **Continuous Deployment (CD)** into the ML pipeline, making sure those models are seamlessly updated and deployed into production environments.

MLOps can automate the process based on predefined triggers like performance drops or the introduction of new data.

Version Control:

MLOps frameworks track different versions of the model, datasets, code, and parameters. This ensures that models are reproducible, which is crucial when troubleshooting issues or comparing model versions.

Experimenting & Training:

Aws SageMaker training, MLFlow and **Kubeflow** allow for tracking the lineage of models, datasets, and their performance, enabling teams to revert to previous versions when necessary

Scale:

Carsdepo, which operates internationally and needs to process and deploy models at scale. A good system will enables scalable deployment.

Monitoring:

SageMaker Model Monitor and **CloudWatch** can track the accuracy of predictions, and if model performance degrades, trigger alerts or even automatically start retraining pipelines.

Collaboration:

Data scientists, developers, and operations teams. it allows multiple teams to contribute to different stages—data preparation, model development, and deployment, better communication between teams, ensuring that models are efficiently moved from development to production.

Governance:

MLOps provides the ability to audit model predictions and data processing pipelines, ensuring that models meet compliance, ethical standards and industry-specific standards ex: GDPR.

MLOps ensure that the models can evolve over time, respond to changing business needs, and deliver consistent value.

Q3. System design: You must create an ML system that has the features of a complete production stack, from experiment tracking to automated model deployment and monitoring.

For this problem, create an ML system design (diagram)

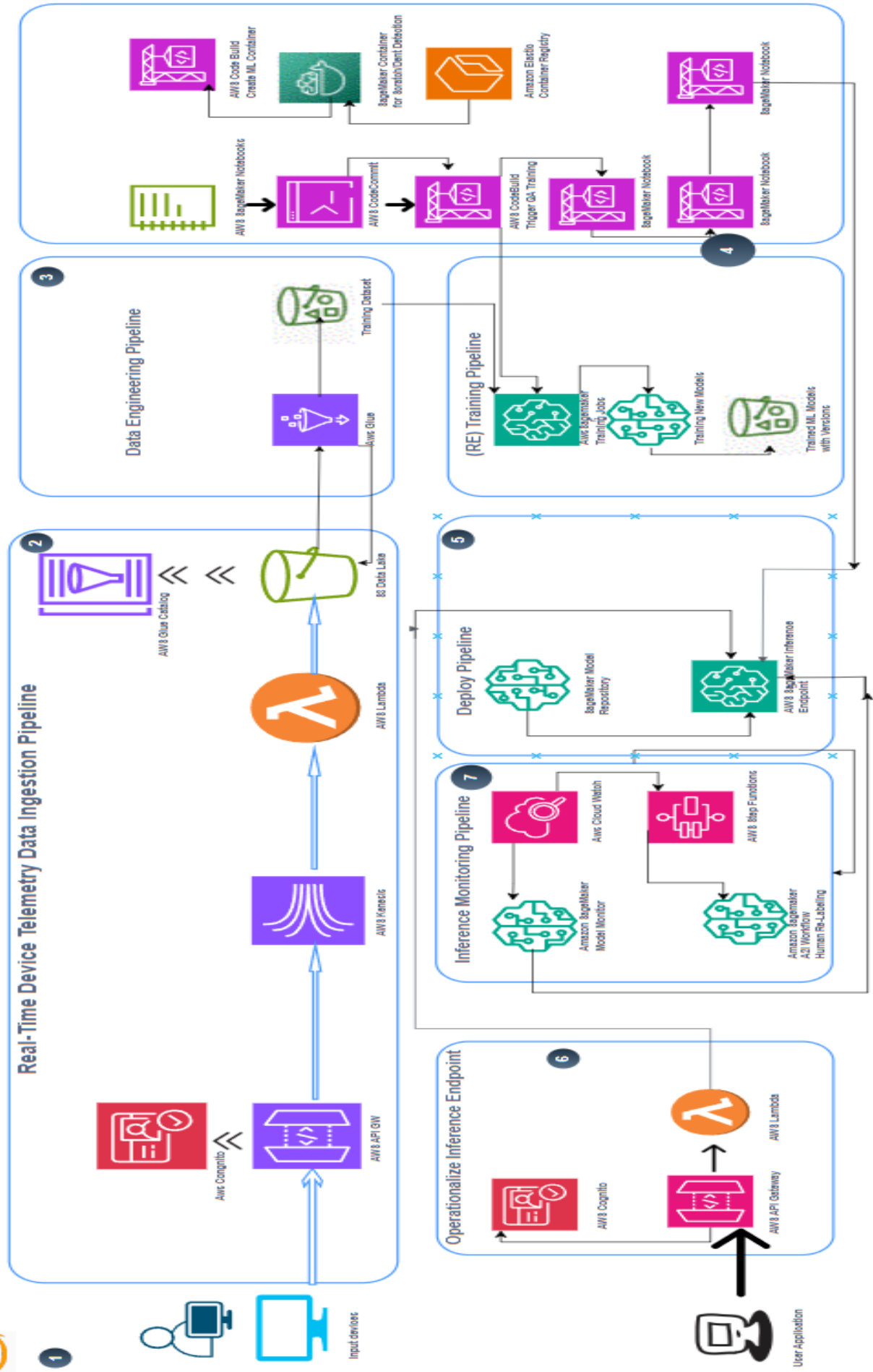
(Note: The MLOps tools you want to use are up to your judgement. You can use open-source tools, managed service tools or a hybrid.

You can use draw.io or any other convenient tool to create the architecture.

You can refer to the [segment link](#) for the architecture created in the customer churn case study.

You can upload the design/diagram as an image in the PDF.)

DrawIO Url: https://app.diagrams.net/#G1dFAVVU-Cwoxpcp_fUnpr4wchZ-mnD3h5#%7B%22pageId%22%3A%22LwC_PyMnSL-ZVbXs3K2y%22%7D



1

1. Device telemetry data is ingested from the field devices on a near real-time basis by calls to the API via Amazon API Gateway. The requests get authenticated/authorized using Amazon Cognito.

2

2. Amazon Kinesis Data Firehose ingests the data in real time and invokes AWS Lambda to transform the data into parquet format. Kinesis Data Firehose will automatically scale to match the throughput of the data being ingested.

3

3. The telemetry data is aggregated on an hourly basis and re-partitioned based on the year, month, date, and hour using AWS Glue jobs. The additional steps like transformations and feature engineering are performed for training the Saccratch / Dent Detection ML Model using AWS Glue jobs. The training data set is stored on Amazon S3 Data Lake.

4

4. The training code is checked in an AWS CodeCommit repo which triggers a Machine Learning DevOps (MLOps) pipeline using AWS CodePipeline. CodePipeline builds the Amazon SageMaker training and inference containers, triggers the SageMaker training job using the specified training dataset, deploys the trained model in the testing environment, and upon approval, deploys the model into production using SageMaker inference endpoints..

5

5. The ML models generated by training jobs are registered in the SageMaker Model Repository. The deploy pipeline selects the best ML model to deploy using SageMaker hosting.

6

6. Classify if the telemetry data is an anomaly or not via HTTP(S) API using Amazon API Gateway and Lambda functions. The Lambda function invokes the SageMaker endpoint to predict the anomaly.

7

7. The inference quality is monitored using SageMaker Model Monitor. The requests with ambiguous prediction scores are sent for re-labeling using Amazon CloudWatch events, triggering the SageMaker A2I workflow using AWS Step Functions.

Q4. System design: After creating the architecture, please specify your reason for choosing the specific tools you chose for the use case.

(Note: The MLOps tools you want to use are up to your judgement. You can use open-source tools, or managed service tools or a hybrid.)

The architecture for the car image detection pipeline employs various AWS services, each chosen for specific capabilities that align with the use case requirements. Here is a detailed explanation of the selected tools:

Amazon Kinesis Data Streams:

This service is ideal for handling real-time data ingestion from field devices, providing scalability and low-latency processing.

Amazon Kinesis Data Streams: This service is ideal for handling real-time data ingestion from field devices, providing scalability and low-latency processing.

Data Engineering

AWS Glue: Selected for its serverless ETL capabilities, which simplify data preparation and transformation, making it easier to prepare data for machine learning.

Machine Learning DevOps

AWS CodeCommit: Hosts source code repositories for version control.

AWS CodePipeline: Automates the build, test, and deployment phases of ML models.

AWS CodeBuild: Compiles source code and runs tests.

AWS CodePipeline: Facilitates continuous integration and delivery, automating the build, test, and deployment phases of machine learning models.

Amazon SageMaker: Central to the architecture, SageMaker offers comprehensive tools for building, training, and deploying ML models at scale. It supports built-in algorithms like Random Cut Forest (RCF) for dent/scratch detection.

Model Deployment and Monitoring

Amazon SageMaker Endpoint: Used to deploy models in a scalable environment, enabling real-time predictions.

Amazon CloudWatch: Provides monitoring and logging capabilities, essential for tracking model performance and operational metrics.

Job Training

AWS Batch:

Manages batch processing jobs efficiently, suitable for retraining models with new data inputs.

AWS Step Functions:

Orchestrates complex workflows by integrating various AWS services, ensuring seamless. By orchestrating tasks in a serverless manner, Step Functions enable scalable execution of complex workflows without manual intervention. It provides built-in error handling and retries, which helps maintain the robustness of the pipeline by managing failures gracefully.

AWS ECR:

Container Storage: ECR stores Docker images that are used to package ML models and dependencies. This ensures consistency and portability across different environments.

Integration with SageMaker: It provides a secure and scalable repository for storing images that SageMaker can pull when deploying models.

Amazon SageMaker Containers:

Model Deployment: Containers encapsulate the ML models and runtime environments, enabling SageMaker to deploy models efficiently.

Scalability and Management: They allow for scalable model serving, ensuring that predictions can be handled efficiently as demand fluctuates.

Amazon SageMaker training jobs :

These are crucial in MLOps for several reasons:

Scalability: They allow for scalable training of machine learning models on large datasets using distributed computing resources. It automates the model training process, integrating seamlessly with other AWS services to streamline the workflow. Supports various ML frameworks and algorithms, enabling customization and experimentation. Optimizes

resource usage and reduces training time with features like automatic model tuning.

AWS API Gateway:

API Management: It provides a secure and scalable interface for accessing machine-learning models deployed on Amazon SageMaker. Offers features like authentication and authorization to protect model endpoints. Manages traffic efficiently, ensuring that the system can handle varying loads without performance degradation.

AWS Cognito provides:

User Authentication: It provides secure user sign-up and sign-in capabilities, ensuring that only authorized users can access the MLOps platform.

Access Control: Manages user permissions and roles, allowing fine-grained access to different parts of the MLOps pipeline.

Integration: Easily integrates with API Gateway and other AWS services to secure endpoints and manage user sessions.

AWS A2I :

Used to integrate human review into machine learning workflows. It allows you to set up human reviews for model predictions, ensuring higher accuracy and handling edge cases where human judgment is needed. This can be particularly useful in scenarios where model predictions have high uncertainty or need validation.

Amazon SageMaker Model Repository:

It stores and organizes trained models, making it easy to manage different versions and track changes. Facilitates collaboration among data scientists and developers by providing a centralized location for models. Simplifies the deployment process by allowing seamless integration with SageMaker endpoints for real-time inference. Supports scalable model serving, ensuring that models can be deployed and accessed efficiently as demand grows.

Include Annotation later when it's available:

When annotations or labels become available later, the Glue Data Catalog helps by serving as a centralized repository to store and retrieve this metadata

efficiently. When annotations are updated in the Glue Catalog, they become accessible to SageMaker training and inference jobs. They can be included in the data pipeline for real-time or batch predictions without having to reconfigure the entire workflow.

Schema Evolution and Data Versioning: If the schema of the dataset changes (for example, when new columns for annotations are added), AWS Glue Data Catalog supports schema evolution. It automatically updates the schema definition, ensuring that the ML models can access the latest schema with new annotation fields. **AWS Glue jobs** can be used to run queries on the data, including the newly added annotations. For example, if annotations like car damage labels (scratches, dents) are stored in the Glue Catalog, you can query only the annotated data for model retraining or validation.

AWS Glue Data Catalog supports partitioning, which allows you to organize annotated datasets by different attributes (e.g., date, damage type). This helps optimize data access for predictions by allowing models to fetch only the relevant partitions of the data based on the most recent annotations.

Example Use Case:

Let's say you are building a car damage detection model, and over time, new annotations (e.g., more images labelled with "dent" or "scratch") become available. You can update the Glue Catalog with these annotations, which will then be accessible for both:

Model retraining: Ensuring that the model is always trained on the latest annotated data.

Model predictions: Ensuring the model has the most up-to-date information about features and annotations during inference.

This architecture leverages AWS managed services to deliver a scalable, reliable, and efficient solution for real-time anomaly detection in car images. The integration of Amazon SageMaker is pivotal due to its robust machine learning capabilities and seamless interaction with other AWS services. This design ensures that the system can handle large volumes of data while maintaining flexibility and operational excellence's operation of the entire pipeline.

Q5. Workflow of the solution:

1. Data and Model Experimentation

Tools: Amazon SageMaker

Model Selection: Choose algorithms like Random Cut Forest (RCF) to detect anomalies such as dents and scratches in car images.

Data Preparation:

Collect images of cars with various lighting conditions and angles.

Preprocess images to enhance features related to surface damage, such as adjusting contrast and brightness.

Feature Engineering:

Extract relevant features from images, like edges and textures, that highlight dents and scratches.

Hyperparameter Tuning:

Experiment with different hyperparameters to optimize model performance, such as learning rates and tree depths.

Validation:

Use a validation set of car images to test model accuracy in detecting surface anomalies.

Iterate on model improvements based on validation results.

This phase is crucial for developing effective models that can accurately identify dents and scratches in car images under various conditions.

2. Automation of Data Pipeline

Tools: Amazon Kinesis Data Streams, AWS Lambda, AWS Glue, AWS Glue Data Catalog

Amazon Kinesis Data Streams:

Ingest real-time telemetry data from network edge field devices.

Ensure low-latency processing to handle continuous data flow.

AWS Lambda:

Trigger real-time processing tasks as data arrives.

Perform initial transformations and filtering of incoming data.

AWS Glue:

Execute ETL (Extract, Transform, Load) operations to prepare data for analysis.

Clean and format data to ensure consistency and quality.

AWS Glue Data Catalog:

Maintain a centralized metadata repository for all ingested data.

Enable easy discovery and access to datasets for downstream processes.

This automated pipeline ensures efficient handling and preparation of data for machine learning tasks, enabling seamless integration with other components of the MLOps

3. Automation of Training Pipeline

Tools: AWS CodeCommit, AWS CodePipeline, AWS CodeBuild, Amazon SageMaker

AWS CodeCommit:

Store and manage source code repositories for model development.

Enable version control for collaborative work among data scientists.

AWS CodePipeline:

Automate the build, test, and deployment process for machine learning models.

Integrate seamlessly with other AWS services for a streamlined workflow.

AWS CodeBuild:

Compile source code and run tests to ensure model integrity.

Provide continuous integration to detect issues early in the development cycle.

Amazon SageMaker:

Train models using distributed computing resources to handle large datasets efficiently.

Utilize built-in algorithms and custom scripts for model training.

Optimize resource usage and reduce training time with features like automatic model tuning.

4. Automation of Inference Pipeline

Tools: Amazon SageMaker Endpoints, Amazon API Gateway

Amazon SageMaker Endpoints:

Deploy trained models for real-time inference on new car images.

Scale automatically to handle varying loads and ensure low-latency predictions.

Amazon API Gateway:

Provide a secure and scalable interface for accessing model predictions.

Manage traffic efficiently, ensuring reliable performance under different conditions.

Enable authentication and authorization to protect model endpoints.

This automated pipeline ensures seamless integration of model predictions into applications, providing real-time insights and maintaining high availability.

API Gateway: Manage secure and scalable access to model predictions.

5. Continuous Monitoring Pipeline

Tools: Amazon CloudWatch, AWS Lambda

Amazon CloudWatch:

Collects and monitors logs and metrics from deployed models.

Sets up alarms to notify when performance metrics deviate from expected thresholds.

Provides dashboards for real-time visualization of model performance and system health.

AWS Lambda:

Processes monitoring data to trigger alerts or automated workflows.

Executes custom logic in response to specific events detected by Cloudwatch.

This pipeline ensures continuous oversight of the model's performance, enabling timely interventions and adjustments to maintain accuracy and reliability.

Actions for Specific Conditions

- Sudden Increase in Drift Due to Poor Lighting

Detection and Response:

Use Cloudwatch to monitor drift.

Trigger retraining via Step Functions if drift is detected.

Initiate manual review or additional data collection if drift exceeds thresholds.

- Additional Annotated Data Available

When additional annotated data becomes available, the pipeline can be enhanced through the following steps:

Detection of New Data:

AWS Glue Data Catalogue: Updates with new annotations, ensuring the metadata repository reflects the latest data.

Data Processing:

AWS Glue: Executes ETL jobs to incorporate new annotations into the existing dataset, preparing it for model training.

Triggering Retraining:

AWS Step Functions: Orchestrates the workflow to initiate a new training job in Amazon SageMaker using the updated dataset.

Model Training:

Amazon SageMaker: Utilizes the additional annotated data to retrain models, improving accuracy and performance.

Deployment and Monitoring:

Deploy updated models to SageMaker Endpoints.

Use Amazon CloudWatch to monitor model performance and ensure the integration of new data enhances prediction quality.

This process ensures that the machine learning models remain accurate and effective by continuously incorporating new insights from additional annotated data.